

Technics SX-WSA1R

System Exclusive Reference

Acoustic Modeling Synthesizer Module

Operating system version 2.0

Contents

1	Overview	3
1.1	Applicability	3
1.2	Ports	3
1.3	Relationship to the published manual	3
1.4	Verification	3
1.5	Conventions	4
2	Message framing	5
2.1	Identifiers	5
2.2	Termination	5
2.3	Length limit	5
3	The Technics message set	6
3.1	Message families	6
3.2	The model triple	6
3.3	Short messages	6
3.4	Tempo	7
3.5	Parameter families	7
4	Bulk dump	8
4.1	Categories and what each transmits	8
4.2	Data message	9
4.3	Payload encoding	9
4.4	Checksum	9
4.5	Requesting a dump	10
4.6	Opening a transfer	10
4.7	Acknowledgement and retry	10
4.8	Continuation	10
4.9	Closing	10
4.10	Receiving	11
5	What a block contains	12
5.1	Signatures	12
5.2	Names	12
6	Parameter messages	13
6.1	Parameter area	13
6.2	Parameters and the bulk dump are the same bytes	13
6.3	What a parameter accepts	14
6.4	Names	17

6.5	Whole-area request	17
6.6	A third region	17
7	Universal System Exclusive	18
7.1	Recognised messages	18
7.2	Messages not recognised	18
7.3	Practical consequences	18
7.4	The identifier is checked once	19
8	Settings that affect System Exclusive	20
8.1	The EXCLUSIVE filter	20
8.2	Which sockets carry what	21
8.3	Memory protect	21
9	Errors	22
9.1	The messages	22
9.2	ERROR 40 — wrong identification	22
9.3	ERROR 41 — reception	22
9.4	ERROR 42 — transmission	23
9.5	Interference from other traffic	23
10	SX-WSA1 and SX-WSA1R	24
11	Relationship to other Technics instruments	26
11.1	What is shared	26
11.2	What distinguishes the products	26
11.3	Dump regions differ	26
12	Worked examples	28
12.1	A parameter, written and read back	28
12.2	A frame from a real instrument	28
13	Scope of this edition	29

Chapter 1

Overview

The Technics SX-WSA1R exchanges data over MIDI System Exclusive. This reference describes the messages it sends and accepts: their byte layout, the bulk-dump transfer protocol, the universal messages it honours, and what it reports when a transfer fails.

1.1 Applicability

This reference describes operating system version **2.0**. An earlier version 1 exists; its behaviour may differ and is not covered.

1.2 Ports

The instrument has two sets of MIDI sockets, MIDI 1 and MIDI 2. Ordinary performance data is handled on both. **Bulk dump transfers use MIDI 1 only** — a dump offered on MIDI 2 is not received, and the instrument sends dumps only from MIDI 1.

1.3 Relationship to the published manual

Technics' own guide documents SYSEX BULK DUMP as a feature: which menu rows exist, that the SEND button starts a transfer, that the MIDI 1 sockets must be used, and three error numbers. It states no byte layouts, and the instrument shipped without a MIDI implementation chart. This reference supplies what the guide leaves out, and follows it wherever both describe the same thing — with one exception, the meaning of ERROR 42, set out in chapter 9.

1.4 Verification

The message layouts, the payload encoding and the checksum rule in this reference have been checked against a bulk dump captured from a real instrument: 2883 messages carrying a SOUND and COMBINATION transfer. Of these, 2876 carry a checksum, and all 2876 verify under the rule given in chapter 3. Every transfer in the capture delivered exactly the byte count its header declared.

The parameter placement in chapter 6 has been checked against the same capture. The SYSTEM, PART & MIDI category was reassembled from it and each parameter's own field read out of the result, for all 32 parts: 1833 fields, of which 1832 hold a value inside the range the instrument accepts for that parameter. Run against a plausible but wrong placement —

one uniform stride for all 32 parts, instead of the two runs chapter 6 describes — the same check puts 69 fields out of range, so it is capable of rejecting a wrong map.

Two fields in that capture label themselves. Every part's MIDI channel holds its own part number, so the 32 parts spell out an unbroken ramp from 0 to 31 — which the wrong placement interrupts at part 8, the first part past the gap. Every part's pan holds 64, the value the instrument's own scale displays as CTR. Those two names in chapter 6 therefore rest on a real dump as well as on the instrument's screen.

1.5 Conventions

Byte values are hexadecimal. A field written *nn* takes one of the values listed for it. Ellipses inside a message stand for a body whose length depends on the message.

Every message begins with F0 and ends with F7, as required by MIDI.

Real-time messages are safe inside a transfer

System Real Time bytes (F8–FF) may be interleaved anywhere inside a System Exclusive message without disturbing it. Conversely, while the instrument is receiving a System Exclusive message it ignores F8 TIMING CLOCK, FA START, FB CONTINUE and FC STOP for transport purposes.

Chapter 2

Message framing

2.1 Identifiers

The instrument accepts exactly two System Exclusive identifiers:

identifier	meaning
50	Matsushita (Technics)
7E	Universal Non-Real Time

A message carrying any other identifier is tracked to its F7 and discarded. No error is reported for it.

There is no device or unit number after the identifier

Many instruments place a device id immediately after the manufacturer identifier. The SX-WSA1R does not. The byte following 50 is already the message-family byte described in chapter 3. A unit field does appear later in some families, but it is not where the convention would put it.

2.2 Termination

A message ends at F7. If any other status byte in the range 80–F6 arrives while a message is open, the message is abandoned and its contents discarded.

2.3 Length limit

A complete message may not exceed 256 bytes counting F0 and F7, that is 254 bytes between them. A longer message is abandoned and, inside a bulk-dump session, reported as a reception error. Dumps sent by the instrument use the full 256 bytes for all but the last message of each transfer.

Chapter 3

The Technics message set

3.1 Message families

After F0 50, the third byte selects a message family. The instrument accepts fourteen values:

21 22 23 24 25 27 28 29 2A 2B 2C 2D 7E 7F

26 is not accepted. The SX-WSA1R's own product identifier is 2D; its outgoing bulk-dump data messages all begin F0 50 2D.

3.2 The model triple

The families 21, 22, 2B, 2C and 2D continue with three fixed bytes:

04 *nn* 11

where *nn* is 00 or 01. A message whose triple the instrument does not recognise is rejected with ERROR 40.

The middle byte names the model, and only on transmission

The keyboard transmits 04 00 11; the rack transmits 04 01 11. The rack substitutes the middle byte as the message leaves, in every 21, 22, 2C and 2D message it sends, and recomputes the checksum of those that carry one, so the substitution is invisible in the checksum. A bulk dump captured from a real rack carries 04 01 11 throughout.

Reception is model-blind. Both values are accepted by both models, on every family that carries the triple, and reach the same handler; a message is never refused, ignored or treated differently because of the middle byte. A receiver must therefore accept both, and a file dumped from one model loads into the other.

3.3 Short messages

These messages are complete as shown. They form the control vocabulary of a bulk-dump session.

message	meaning
F0 50 23 7E F7	acknowledge
F0 50 24 7E F7	not acknowledged
F0 50 27 7E F7	end of category
F0 50 28 7E F7	end of dump
F0 50 29 7E F7	abort
F0 50 2A 7E F7	destination full
F0 50 21 04 <i>nn</i> 11 F7	enquiry
F0 50 22 04 <i>nn</i> 11 F7	enquiry reply

3.4 Tempo

The family 25 carries a tempo and nothing else:

F0 50 25 *lo hi* F7

Six bytes. Unlike every other Technics family it carries **no model triple, no unit field and no checksum**, and any bytes sent beyond the fifth are ignored. The tempo is nine bits sent low nibble first: $value = lo + 16 hi$. The accepted range is 40 to 300 beats per minute and a value outside it is treated as 120.

This message is the one thing the EXCLUSIVE filter governs; see chapter 8.

See chapter 10 — the tempo message is one of two things the rack does not support.

3.5 Parameter families

The families 2B and 2C carry individual parameter traffic rather than bulk transfer. Both address one parameter space, which chapter 6 sets out and enumerates; chapter 12 works a write and a read through byte by byte.

Chapter 4

Bulk dump

Bulk dump transfers the instrument's internal data as System Exclusive. The instrument is silent for the duration, and the transfer uses the MIDI 1 sockets only.

4.1 Categories and what each transmits

The SYSEX BULK DUMP screen offers five categories. Each sends one or more data messages at fixed dump addresses.

category	address bytes	length bytes	length	blocks
SYSTEM, PART & MIDI	40 00 00	00 00 20	32	
	40 00 20	00 12 60	2 400	
SOUND	20 00 00	10 00 00	262 144	32
COMBINATION	50 00 00	00 06 00	768	
	50 06 00	05 40 00	90 112	16
SEQUENCER	60 00 00	00 18 00	3 072	
	60 18 00	01 70 00	30 720	
	62 08 00	set at send time		
TOTAL KEYBOARD	the four categories in the order above			

Lengths are in source bytes; on the wire each occupies twice as many, because of the payload encoding described below.

The *blocks* column is internal structure, not extra messages: SOUND sends one header covering all 262 144 bytes, which is 32 repeats of an 8 192-byte layout, and COMBINATION's second block one header covering 90 112 bytes as 16 repeats of 5 632. Those are the instrument's 32 sounds and 16 combinations.

TOTAL KEYBOARD runs the four categories one after another in the order listed, each closed by its own end-of-category message, with a single end-of-dump message after the last.

The SEQUENCER category is the only one whose final message has no fixed length — it carries as much recorded material as the instrument holds.

The dump address is not a memory address

Within one category the addresses are consecutive: a block's address is the previous block's address plus its length. Across categories they are not. SOUND and COMBINATION are held side by side inside the instrument, yet their dump addresses are 786 432 apart; other pairs are the reverse.

Treat the address and length bytes together as a fixed name for one of the eight blocks. The instrument does the same — on reception it compares them against a fixed list and performs no arithmetic on them.

4.2 Data message

F0 50 2D 04 *nn* 11 *a2 a1 a0 l2 l1 l0 data... c sum* F7

nn unit field, 00 or 01.

a2 a1 a0 destination address, three bytes, most significant first, seven bits each: the value is $a_2 \times 128^2 + a_1 \times 128 + a_0$.

l2 l1 l0 length in *source* bytes, encoded the same way.

data the payload, encoded as described below.

c continuation flag: 01 if further messages follow, 00 on the last.

sum checksum.

A message is filled to at most 252 bytes before being sent; longer transfers are split.

4.3 Payload encoding

Each source byte is sent as two message bytes, **high nibble first**:

$$b \longrightarrow (b \gg 4), \quad (b \& 0F)$$

So source byte F1 appears on the wire as 0F 01, and the payload occupies twice as many bytes as the length field states. A receiver rejoins them as $((\text{first} \& 0F) \ll 4) | (\text{second} \& 0F)$.

An odd number of payload bytes is invalid.

4.4 Checksum

Let S be the eight-bit sum of every byte from the identifier 50 through the continuation flag inclusive. Then

$$\text{sum} = (0 - S) \& 7F$$

The leading F0, the checksum byte itself and the trailing F7 take no part in the sum. Equivalently, the sum of every byte after F0 including the checksum is zero modulo 128.

The checksum is not verified for every family

On reception the checksum is checked only for families 7E, 2B, 2C and 2D. For the other accepted families a wrong checksum passes unremarked.

4.5 Requesting a dump

A dump can be asked for over MIDI; the front-panel SEND button is not the only way to start one. The request names a category by its region byte:

F0 50 2B 04 *nn* 11 *area* 00 00 *x x x* F7

<i>area</i>	category requested
40	SYSTEM, PART & MIDI
20	SOUND
50	COMBINATION
60	SEQUENCER

The three bytes in the length position are ignored; any values are accepted. The instrument then runs exactly the transfer the corresponding menu row would have run.

There is no request for TOTAL KEYBOARD: the four categories must be asked for one at a time.

4.6 Opening a transfer

A dump begins with a two-step exchange. Each step is attempted up to three times.

1. The sender transmits F0 50 21 04 00 11 F7 and expects F0 50 22 04 *nn* 11 F7.
2. The sender transmits F0 50 22 04 00 11 F7 and expects F0 50 23 7E F7.

If the exchange succeeds, every subsequent data message must be acknowledged. If all three attempts of the first step fail, **the dump proceeds anyway, without acknowledgements**, at a fixed pace of roughly 51 ms between messages.

4.7 Acknowledgement and retry

With acknowledgements in force, each data message must be answered with F0 50 23 7E F7. Any other reply causes the same message to be retransmitted, up to three attempts, after which the transfer is abandoned.

A reply is awaited for about 5.1 seconds; during the opening exchange the wait is about 2.05 seconds.

4.8 Continuation

Data messages after the first open with F0 50 7E, followed by payload, continuation flag, checksum and F7.

4.9 Closing

At the end of each category the sender transmits F0 50 27 7E F7 and waits for an acknowledgement. At the end of the whole transfer it transmits F0 50 28 7E F7. If the transfer was abandoned after three failed retries it transmits F0 50 29 7E F7.

4.10 Receiving

When the SX-WSA1R is the receiver it answers each data message with `F0 50 23 7E F7` to accept, `F0 50 24 7E F7` to reject, and `F0 50 2A 7E F7` when the incoming data is larger than the destination.

While a bulk-dump session is open the instrument accepts only the `50` identifier; universal messages are not honoured until the session ends.

The instrument must be on the SYSEX BULK DUMP screen to receive

A dump cannot be pushed to an idle instrument. Reception happens only while that screen is displayed; a dump sent at any other time is parsed, discarded, and not reported.

A dump offered on the MIDI 2 input is never received either, at any time. This is what lies behind the published instruction to use MIDI 1 for bulk transfers.

Seven of the eight patterns are pinned on all six bytes, address and length together. Sending a correct address with a shortened length is not a short transfer — it is an unrecognised message, refused with ERROR 41. Only SEQUENCER block 3 has its length read as a number, because that block's size varies. An arbitrary destination cannot be written.

The one length the message itself controls is SEQUENCER part 3, which is capped at 330 752 bytes; beyond that, and whenever the destination fills mid-message, the instrument answers `F0 50 2A 7E F7` and reports ERROR 21.

Memory-full is treated as end-of-dump

When the SX-WSA1R is *sending* and the receiver answers `F0 50 2A 7E F7` to report that its memory is full, the instrument treats that exactly as it treats `F0 50 28 7E F7`, end of dump. It stops, and it reports success. A sender that needs to know whether the data arrived cannot rely on the instrument to notice.

Chapter 5

What a block contains

This reference describes the protocol that carries a dump. The meaning of the data inside a block is outside its scope, with one exception worth stating: every block opens with a fixed signature, and two of them carry readable names. Together these let a program check a stored dump before sending it back, and list what is in it, without understanding the rest.

5.1 Signatures

The bytes below are the first bytes of the reassembled block — after the payload has been rejoined from its nibbles, not as they appear on the wire.

block	first bytes	as text
SYSTEM, PART & MIDI 1	5A 5A 01 00 57 41 30 00	ZZ..WA0.
SYSTEM, PART & MIDI 2	78 10 ...	x.
SOUND	57 53 41 20 53 4F 55 4E	WSA SOUND RAM S0
COMBINATION 1	5A 5A 5A 5A 00 00 57 53	ZZZZ..WSA1
COMBINATION 2	78 10 ...	x.

A file whose first bytes do not match the block it claims to be is not a dump of that category.

5.2 Names

The SOUND block continues, immediately after its sixteen-byte signature, with sixteen-byte names in the instrument’s own character set, padded with spaces. The second COMBINATION block and the second SYSTEM, PART & MIDI block use a different framing: a two-byte prefix, then a sixteen-byte name, then two further bytes.

A librarian can therefore show the user what a stored file holds — “SWEEP PAD”, “Space Infinity” — without decoding anything else.

Reassembly is self-checking

A block reassembled correctly comes out at exactly the length its own header declared. A wrong nibble order, or a payload taken with its trailing flag and checksum still attached, does not. Checking the length is a cheap way to confirm a receiver is unpacking correctly before trusting anything it produces.

Chapter 6

Parameter messages

Two message families address individual parameters rather than transferring blocks. Family 2C writes; family 2B requests. Both share one address space.

F0 50 2B 04 *nn* 11 *r s p* ...F7

The three bytes after the model triple select what is addressed.

6.1 Parameter area

When *r* is 00, *s* selects a block and *p* selects a parameter within it.

<i>s</i>	block	parameters
00	common block 0	16
01	common block 1	6
08	write only	4
10	common block 10	20
11	common block 11	4, plus a further 68 values
20–3F	part 0 to part 31	57 each
60	single parameter	1

The part blocks are regular: *s* is 20 plus the part number, and all 32 parts take the same 57 parameters. A few parameters carry more than one data byte.

Under *s* = 11, values of *p* from 03 upward are admitted in three runs — 20–36, 40–56 and 60–75. Any other value in that block is accepted by the parser and then discarded, producing no error and no effect.

Counting the part block once, the area holds 108 distinct parameters.

6.2 Parameters and the bulk dump are the same bytes

The individual parameters and the SYSTEM, PART & MIDI bulk dump are two views of one object. That dump carries addresses 100000 to 100980, and every parameter in the 00 area is one byte — or a field of one byte — inside it.

This matters in both directions. An editor that writes parameters and a librarian that restores dumps are competing for the same storage, so a dump restored after a parameter write discards that write. It also gives a way to confirm any row of the tables below without opening the instrument: dump the category, change the one parameter, dump again, and the two dumps differ at the stated address and nowhere else.

Where a part's record sits

Each part occupies a 40-byte record, and the offsets in table 6.1 are relative to the start of that record. The records are in two runs:

part	record begins at
0 to 7	$1000A2 + 40 \times \text{part}$
8 to 31	$1002E2 + 40 \times (\text{part} - 8)$

The part records are not one array

The obvious formula, $1000A2 + 40 \times \text{part}$, is correct for the first eight parts and wrong for the other twenty-four. A 40-byte gap after part 7 holds common-block storage, and the parts resume above it. Code that assumes a single stride will read and write the wrong part for parts 8 upward, and will corrupt two common blocks on the way past.

6.3 What a parameter accepts

Each parameter has a fixed data length and a range of accepted values. A value outside the range is rejected without an error message. Where the tables give a range of 0 or 1 the parameter is a single bit, and several such parameters usually share one byte of the dump — so a dump editor changing one of them must preserve the others in the same byte.

Three parameters accept an enumerated set rather than a range; sending any other value has no effect.

A dump may hold values a parameter message would refuse

The accepted values in the tables are what the instrument takes from a 2C write. They are not a guarantee about what a stored byte contains. In a dump taken from a real machine, one field — common block 00, parameter 08 — holds 00 although a write of 00 to it would be rejected.

So a librarian should not validate a dump by checking it against these ranges, and should not assume a value read out of a dump can be written back one parameter at a time.

Table 6.1: The 57 parameters of a part block. *p* is the third address byte; the offset is the byte's position inside that part's record in a SYSTEM, PART & MIDI bulk dump.

<i>p</i>	data	offset	bits	values	name
00	3	00	all	0–127	
03	1	03	7F	0–127	VOL (LVL)
04	1	05	7F	0–127	EFF1
05	1	06	7F	0–127	EFF2
06	1	07	7F	0–127	REV
07	1	08	7F	0–127	PAN
08	1	09	7F	28–100	
09	1	0A	all	0–255	
0A	1	0B	7F	0–12	

<i>p</i>	data	offset	bits	values	name
10	1	26	0F	0–1	
14	1	0C	08	0–1	
20	1	25	all	0–48	
28	1	27	7F	0–127	
29	1	28	7F	0–127	
2A	1	29	7F	0–127	
2B	1	2A	7F	0–127	
30	1	23	all	0–1	
31	1	24	all	enumerated, 4 values	
40	1	0D	20	0–1	INT (internal sound)
41	1	0D	1F	0–31	MIDI channel
42	1	0D	40	0–1	
43	1	0D	80	0–1	
48	1	2F	10	0–1	
49	1	30	80	0–1	
4A	1	30	04	0–1	
50	1	33	10	0–1	
51	1	34	80	0–1	
52	1	34	04	0–1	
53	1	34	10	0–1	
54	1	34	20	0–1	
55	1	33	40	0–1	
56	1	34	02	0–1	
57	1	36	10	0–1	
58	1	36	01	0–1	
59	1	36	02	0–1	
5A	1	36	04	0–1	
5B	1	36	08	0–1	
5C	1	34	01	0–1	
5D	1	36	20	0–1	
5E	1	33	20	0–1	
60	1	0E	all	0–129	
61	2	—	all	0–129	
63	1	11	all	0–129	
64	1	12	all	0–129	
65	1	14	all	0–129	
66	1	13	all	0–129	
68	1	37	all	28–100	
70	1	2B	40	0–1	
71	1	2C	02	0–1	
72	1	2E	10	0–1	
73	1	2E	01	0–1	
74	1	2E	02	0–1	
75	1	2E	04	0–1	
76	1	2E	08	0–1	
77	1	2C	01	0–1	
78	1	2E	20	0–1	
79	1	2B	20	0–1	

Table 6.2: The parameters of the common blocks. The address is the one a SYSTEM, PART & MIDI bulk dump carries for the same byte.

<i>s</i>	<i>p</i>	data	dump address	bits	values	name
00	00	1	not dumped	01	0–1	
00	01	1	not dumped	02	0–1	
00	08	1	10094A	all	48–192	
00	10	1	10095A	0F	0–9	
00	11	1	10095C	7F	0–127	
00	12	1	10095F	all	0–10	
00	13	1	100960	7F	0–127	
00	20	1	10094E	all	enumerated, 4 values	
00	30	1	1008E4	0F	0–2	
00	31	1	1008E4	F0	0–2	
00	32	1	100907	04	0–1	
00	33	1	100907	02	0–1	
00	34	1	100907	01	0–1	
00	35	1	100907	08	0–1	
00	36	1	100907	10	0–1	
00	40	2	—	7F	0–127	
01	00	1	100935	0F	0–2	
01	01	1	100935	F0	0–1	
01	02	1	100936	1F	0–15	
01	03	1	100936	20	0–1	
01	04	1	100932	03	0–1	
01	05	1	100932	08	0–1	
08	00	1	—	—	0–6	
08	08	1	—	F0	0–1	
08	10	1	—	—	0–127	
08	11	3	—	—	0–1	
10	00	1	1002B2	7F	28–100	OCT
10	10	1	1002A3	80	0–1	
10	11	1	1002A2	all	enumerated, 15 values	scale tuning type
10	12	1	1002A3	0F	0–11	
10	13	1	1002A4	all	0–255	user scale, note 1
10	14	1	1002A5	all	0–255	user scale, note 2
10	15	1	1002A6	all	0–255	user scale, note 3
10	16	1	1002A7	all	0–255	user scale, note 4
10	17	1	1002A8	all	0–255	user scale, note 5
10	18	1	1002A9	all	0–255	user scale, note 6
10	19	1	1002AA	all	0–255	user scale, note 7
10	1A	1	1002AB	all	0–255	user scale, note 8
10	1B	1	1002AC	all	0–255	user scale, note 9
10	1C	1	1002AD	all	0–255	user scale, note 10
10	1D	1	1002AE	all	0–255	user scale, note 11
10	1E	1	1002AF	all	0–255	user scale, note 12
10	20	1	—	1F	0–17	
10	21	1	1002B3	3F	0–48	
10	22	1	—	1F	17–26	

<i>s</i>	<i>p</i>	data	dump address	bits	values	name
10	23	1	1002B5	3F	0–48	
11	00	1	100034	01	0–1	
11	01	1	1002B7	0F	1–4	
11	02	1	1002B7	F0	1–4	
11	20	1	—	—	0–0	
60	00	1	100905	7F	0–127	

parameter	accepted values	count
00 00 20	00 40 41 42	4
00 10 11	00 40 41 42 03 04 06 10 11 12 13 14 15 16 80	15
00 20 31	00 02 03 04	4

6.4 Names

The name column is filled only where the instrument itself or its published manual names the byte. A name marked † is a match on value range or bit count rather than a statement by the machine. Blank means this edition does not know what the parameter is; its address, length and accepted values are still correct, and are enough to read it out of the instrument and watch what changes.

One parameter does not store what it is sent: 20 05 accepts 0–127 and passes the value through a substitution before storing it, so reading it back does not return what was written.

6.5 Whole-area request

Family 2B also requests an entire category, as described in chapter 4. Here *r* takes the region byte rather than 00.

6.6 A third region

Both families accept *r* values of 10, 18 and 19, which carry an address and a count. What they address is not covered by this edition.

Chapter 7

Universal System Exclusive

7.1 Recognised messages

The instrument honours two universal messages, and transmits both.

message	meaning
F0 7E 7F 09 01 F7	General MIDI System On
F0 7E 7F 09 02 F7	General MIDI System Off

Turning General MIDI on restricts the sounds and operations available and may rearrange the percussion map. General MIDI System Off has no effect if General MIDI is already off.

A received message takes effect at once. The same change made at the front panel does not: the panel asks for a confirmation first, and the user can decline it. A controller sending General MIDI System On is therefore doing something the user cannot be offered the chance to refuse, and the mode change it causes is one the instrument warns will make material recorded in the other mode play back wrongly.

Both messages are also written into Standard MIDI Files produced by the instrument, as a five-byte System Exclusive event at delta time zero.

7.2 Messages not recognised

A controller expecting any of the following will receive no response:

message	
F0 7F 7F 04 01 <i>ll mm</i> F7	Master Volume
F0 7E <i>dd</i> 06 01 F7	Identity Request
F0 7E 7F 09 03 F7	General MIDI 2 System On

The instrument does not implement Identity Reply, so it cannot be auto-detected by a controller that relies on it. The entire Universal Real Time identifier 7F is unrecognised.

7.3 Practical consequences

A General MIDI message sent during a bulk dump breaks the dump. While a transfer is open the instrument accepts only messages beginning F0 50. One beginning F0 7E is treated as a reception failure: the transfer is abandoned and ERROR 41 is reported. General MIDI mode is left unchanged.

The instrument announces its own changes. Entering or leaving General MIDI mode makes it transmit the matching message, for a change made at the front panel as well as one made by loading a song. Neither direction is affected by the EXCLUSIVE filter.

The announcement stops for good once you send one

The interlock that stops the instrument echoing a General MIDI message back is armed by the first such message it receives after power-on, and is never disarmed. Once a controller has sent either message, the instrument stops announcing its own General MIDI changes for the rest of the session. A controller that tracks the instrument's mode by listening for these messages will silently stop being told.

General MIDI state travels in a dump. It is carried in the second SYSTEM, PART & MIDI block, so restoring that category restores the General MIDI setting with it. A dump taken in General MIDI mode is not the same file as one taken out of it.

7.4 The identifier is checked once

Once 50 or 7E has been accepted the instrument does not look at it again; the third byte alone selects the message. F0 50 7F 09 01 F7 is therefore acted on as General MIDI System On despite carrying the Matsushita identifier, and a Technics message sent with 7E in the identifier position works equally well. Send the conventional identifier, but expect a receiver to be lenient.

Nothing else about General MIDI mode changes the System Exclusive implementation: every family is accepted exactly as before, with the same encoding, checksum, handshake and error reporting.

Chapter 8

Settings that affect System Exclusive

8.1 The EXCLUSIVE filter

The MIDI menu's INPUT & OUTPUT FILTER page has eight rows, each a plain ON/OFF switch for one kind of MIDI message: PROGRAM CHANGE, BANK SELECT, PITCH BEND, CONTROL CHANGE, RESET ALL CNT., CHANNEL PRESSURE, SONG SELECT and, last, EXCLUSIVE. There is one such switch and it governs both directions: the same setting is consulted when the instrument sends and when it receives. There is no separate input filter, no per-socket filter and no third state, and the page is the same on both models.

EXCLUSIVE governs the tempo message and nothing else

Despite its name, this switch does not turn System Exclusive on and off. Its entire effect on the messages in this reference is the family 25 tempo message: with EXCLUSIVE set to OFF the instrument neither sends one nor acts on one received, silently in both directions.

A bulk dump transfers normally with EXCLUSIVE set to OFF. Software that cannot complete a transfer should not look here for the cause.

This is worth knowing before telling a user to change the setting: the illustration of that page in Technics' own guide shows EXCLUSIVE switched OFF, so an instrument presenting that page as pictured is already able to send and receive every message in this reference.

Everything below is unaffected by the setting:

function	affected by EXCLUSIVE?
bulk dump, sent or received	no
dump requests	no
the opening enquiry and its reply	no
acknowledgements and the session control messages	no
parameter messages, and the replies and refusals to them	no
General MIDI System On / Off, sent or received	no
error reporting	no
tempo message (family 25)	yes

8.2 Which sockets carry what

The instrument has two sets of MIDI sockets. Only bulk-dump traffic is restricted to the first.

message	accepted on
bulk-dump data, continuation frames, handshake, acknowledgements	MIDI 1 only
dump requests	MIDI 1 only
parameter messages (families 2B, 2C)	MIDI 1 and MIDI 2
tempo message (family 25)	MIDI 1 and MIDI 2
General MIDI System On / Off	MIDI 1 and MIDI 2

MIDI channels are written with the socket set first: 1-01 to 1-16 and 2-01 to 2-16. No channel or device-number setting affects System Exclusive addressing — these messages carry no channel.

8.3 Memory protect

Two write-protect switches are consulted before an incoming message is acted on. A message they block never reaches the transfer machinery, and the instrument shows its own write-protection prompt rather than a System Exclusive error. A transfer that stops with that prompt on screen has not failed as a transfer.

Chapter 9

Errors

Transfer failures are reported on the SYSEX BULK DUMP screen. A System Exclusive message arriving while any other screen is displayed produces no visible error.

9.1 The messages

screen	meaning
COMPLETED!	the transfer finished
ERROR 40!	the message is addressed to a different product
ERROR 41!	reception failed
ERROR 42!	transmission failed
ERROR 21!	the destination is full

Not every failure is a 40, 41 or 42

When incoming data is larger than the destination area, the instrument reports ERROR 21! MEMORY FULL. Software that recognises only the three documented System Exclusive errors will mis-read a truncated restore as something else.

9.2 ERROR 40 — wrong identification

Raised when the message family byte is accepted but the bytes that follow it do not match the product. For a message addressed to the SX-WSA1R this means the model triple was not 04, then 00 or 01, then 11.

9.3 ERROR 41 — reception

The general reception failure. It covers:

- a message inside a session that does not begin F0 50;
- a status byte other than F7 arriving inside an open message;
- a framing, parity or overrun error on the MIDI input, or loss of active sensing;
- no reply within the reply window, including the acknowledgement that each data message must receive;

- a message longer than 254 bytes;
- an odd number of payload bytes;
- a continuation flag that is neither 00 nor 01;
- a checksum mismatch.

9.4 ERROR 42 — transmission

Raised when an outgoing transfer stalls inside the instrument: the send path waits for one block to be handed over internally, and reports ERROR 42 when that wait times out after about a second. In version 2.0 this is its only cause.

ERROR 42 is not a report about the receiving device

The published manual describes ERROR 42 as an error during transmission, with the data not having been received correctly. That is not what the instrument tests. The condition it reports is internal to the sending machine and can arise with nothing connected to the MIDI output at all.

A genuine failure at the far end looks different. No acknowledgement within the reply window, a framing, parity or overrun error on the input, and a transfer abandoned after its three retries all raise ERROR 41. Software diagnosing a failed transfer should read ERROR 41, not ERROR 42, as the sign of a problem on the cable or at the receiver.

9.5 Interference from other traffic

While a bulk-dump session is open, unrelated System Exclusive traffic arriving on the MIDI input can set the error state, and will then be reported when the session ends. Keep other sources off the MIDI 1 input for the duration of a transfer.

Chapter 10

SX-WSA1 and SX-WSA1R

The SX-WSA1 keyboard and the SX-WSA1R rack module run the same program and speak the same System Exclusive protocol. Two features are nonetheless available on the keyboard and not on the rack, in both directions, and the two models put different bytes in the model triple. Nothing else in the System Exclusive implementation depends on which model it is: the framing, the checksum, the handshake, the error reporting, the parameter address space and every other bulk-dump category are identical.

	SX-WSA1 (keyboard)	SX-WSA1R (rack)
message framing, checksum, handshake	yes	yes
SOUND bulk dump	yes	yes
COMBINATION bulk dump	yes	yes
SYSTEM, PART & MIDI bulk dump	yes	yes
dump requests	yes	yes
General MIDI On / Off	yes	yes
model triple transmitted	04 00 11	04 01 11
model triples accepted	both	both
SEQUENCER bulk dump	yes	no
tempo message (family 25)	yes	no

On the rack, a SEQUENCER dump sends nothing and says nothing

The rack's SYSEX BULK DUMP menu has no SEQUENCER row: it is not drawn, and the soft key that would select it does nothing. The category can still be reached over MIDI, by a dump request addressed to 60 00 00, and as part of TOTAL KEYBOARD. In both cases nothing is transmitted — not even the end-of-category message that closes every other transfer. A librarian waiting for data will simply time out.

On the rack the transfer-progress screen likewise lists only SYS,PART&MIDI, SOUND and COMBINATION.

Technics' own guide points the same way: the photograph it prints of the SYSEX BULK DUMP menu, the one showing all five rows, is captioned (WSA1) — the marker the guide uses throughout for what belongs to the keyboard.

The tempo message is refused in both directions on the rack: it will neither act on one received nor send one.

Telling the two apart over MIDI

The middle byte of the model triple in anything the instrument transmits is 00 on the keyboard and 01 on the rack; an identity enquiry (F0 50 21 04 00 11 F7 or F0 50 22 04 00 11 F7) is answered with the model's own value, whichever value the enquiry itself carried. A second, behavioural test is to request a SEQUENCER dump and see whether anything arrives.

Which machine is which

The two behaviours are selected by a hardware input read once at power-on; the program names neither model. The identification rests on two outside sources that agree.

The service documentation for that input identifies the machine that reads the second value as the rack. Independently, Technics' own user guide marks the features the rack lacks: its sequencer chapter is headed SEQUENCER (WSA1), and fifty-nine passages across the guide carry the same (WSA1) marker for keyboard-only functions. A model with no sequencer has nothing to put in a SEQUENCER dump and no use for a tempo message, which is exactly the pair of System Exclusive functions that machine refuses.

Chapter 11

Relationship to other Technics instruments

The message engine described in this reference is not unique to the SX-WSA1R. At least two other Technics instruments implement the same protocol: the SX-KN5000 and the SX-KN1500.

11.1 What is shared

All three use the 50 identifier, the same message families, the same short control messages, the same payload encoding and the same checksum rule. The fixed messages are byte-for-byte identical:

```
F0 50 23 7E F7  F0 50 24 7E F7  F0 50 27 7E F7  F0 50 28 7E F7  F0 50 29 7E
F7  F0 50 2A 7E F7
```

The error mapping is also nearly common: of the thirty-four status values, thirty-two produce the same report on the SX-WSA1R and the SX-KN5000.

11.2 What distinguishes the products

The model triple is the discriminator. A message whose triple does not match the receiving instrument is refused with ERROR 40.

instrument	model triple
SX-WSA1R	04 00 11, 04 01 11
SX-KN5000	01 28 12
SX-KN1500	01 24 11, 01 25 11, 01 26 11

A librarian that supports more than one of these instruments needs only to vary the triple and the dump region addresses; the framing, handshake, encoding and checksum are common code.

11.3 Dump regions differ

The dump region addresses are per-product. The SX-KN5000 and SX-KN1500 share eleven of their thirteen region and length pairs with each other, but the SX-WSA1R shares none of its eight with either — its categories are its own.

The SX-WSA1R does not accept the GS-style messages its siblings do

The SX-KN5000 and SX-KN1500 additionally accept a small set of messages in the 42 12 parameter form, including the sequence 42 12 40 00 7F 00 41 used elsewhere in the industry as a reset. **The SX-WSA1R accepts none of them.** Software that resets a KN-series instrument that way and assumes the SX-WSA1R will follow will find it does not respond.

Chapter 12

Worked examples

Three things in this format are easy to get subtly wrong: the seven-bit address encoding, the high-nibble-first payload, and which bytes the checksum covers. The examples below are complete messages, and every byte in them is computed rather than transcribed.

12.1 A parameter, written and read back

The example is part 0's parameter 03, a one-byte value, set to 100.

	message
write	F0 50 2C 04 00 11 00 20 03 00 00 01 06 04 00 41 F7
request	F0 50 2B 04 00 11 00 20 03 00 00 01 00 4C F7
reply	F0 50 2C 04 00 11 00 20 03 00 00 01 06 04 00 41 F7
part 7	F0 50 2C 04 00 11 00 27 03 00 00 01 06 04 00 3A F7
three-byte	F0 50 2C 04 00 11 00 20 00 00 00 03 01 02 00 00 04 00 00 45 F7

The value 100 is 64, which the payload carries as the two bytes 06 04. The three bytes before it are the count, and the 00 after it is the continuation flag. The checksum closes the message.

12.2 A frame from a real instrument

The bytes below open the first data message of a dump recorded from an SX-WSA1R, a transfer of three categories in one session:

F0 50 2D 04 01 11 40 00 00 00 00 20 ...

Reading it by the rules of chapter 4: family 2D, model triple 04 01 11 — the rack's — destination 40 00 00 and length 00 00 20, which together name block 1 of SYSTEM, PART & MIDI. The length decodes to 32 source bytes and the frame carries 32 of them.

The payload decodes to 5A 5A 01 00 57 41 30 00..., the signature chapter 5 gives for that block.

That block is the one chapter 6 maps: a dump of this category can be read against table 6.1 directly, which is the cheapest way to confirm a parameter address without an instrument to hand.

Chapter 13

Scope of this edition

This reference covers message framing, the bulk-dump transfer protocol, the universal messages, and error reporting. The following are not covered.

Parameter meanings. Chapter 6 gives the structure of the parameter address space and the size of each block, but not what each individual parameter controls.

Block contents. Chapter 5 gives each block's signature and where its names are, but not the meaning of the rest of the bytes — the structure of a sound, a combination or a sequence.

Operating system version 1. Everything here describes version 2.0.